Projet Files d'Attente : Simulation Vélib

Modélisation, Simulation et Limites d'un réseau fermé

Sarah Daher et Salomé Bellais

Télécom Paris - Institut Polytechnique de Paris

Avril 2026

1. Modélisation et Hypothèses

Le Modèle

Réseau fermé modélisant la circulation de vélos (clients) entre $M = 5$ stations. L'état du système est le vecteur $x = (e_i, e_{jk})$ où ei= nombre de vélos dans la station i et ejk = nombre de vélos en route de la station j à la station k

Hypothèses :

- **Processus Markoviens :** Arrivées de Poisson (λ_j) et temps de trajets exponentiels ($\mu_{jk} = 1/t_{jk}$).
- **Capacité infinie :** Les stations peuvent accueillir un nombre infini de vélos.
- **Absence de file d'attente :** Un usager est servi immédiatement si un vélo est disponible. Sinon, il repart (perte du client, pas de mise en attente).
- **Réseau fermé :** Le nombre total de vélos N est constant.

2. Analyse Théorique : Modélisation des Flux ($N = 1$)

Principe de l'état stationnaire

À l'état stationnaire, le flux de vélos entrant dans un état (station ou trajet) compense exactement le flux sortant.

Les équations de bilan :

Sur les trajets ($j \to k$) :

- $\pi_j(\lambda_j p_{jk}) = \pi_{jk}\mu_{jk}$

$\Longrightarrow \quad \pi_{jk} = \frac{\lambda_j p_{jk}}{\mu_{jk}}\pi_j$

Dans les stations (i) :

- $$\underbrace{\pi_i \lambda_i}_{\text{Flux sortant}} = \underbrace{\sum_{j \neq i} \pi_j \lambda_j p_{ji}}_{\text{Flux entrant}}$$

2. Analyse Théorique : Résolution et Probabilité

- on pose $\alpha_i = \pi_i \lambda_i$. L'équation des stations devient :

$$\alpha_i = \sum_{j \neq i} \alpha_j p_{ji}$$

- α est le **vecteur propre** de la matrice de transition P^T associé à la valeur propre 1.

2. Calcul de la Probabilité

- On introduit K, la **constante de normalisation** (la somme des probabilités de tous les états vaut 1).
- **Résultat :** La probabilité que la station i soit vide est l'événement complémentaire de la présence du vélo :

$$\mathbb{P}(e_i = 0) = 1 - \frac{\alpha_i / \lambda_i}{K}$$

3. Fonctionnement de la Simulation ($N = 100$) - Part 1

Principe : La simulation repose sur la propriété du **minimum de lois exponentielles** pour déterminer le temps et la nature du prochain événement.

1. Loi du Prochain Saut

Chaque événement potentiel j suit une loi exponentielle de paramètre r_j, défini selon sa nature :

- **Départ** (de la station i vers k) : $r_j = \lambda_i p_{ik} \quad (si\ n_i > 0)$
- **Arrivée** (à la station k depuis la route i) : $r_j = nbvelos_{ik}\mu_{ik}$

Le temps avant le premier événement de la liste est :

$$dt = \min(X_1, X_2, \ldots, X_n) \sim \mathcal{E}xp\left(\sum r_j\right)$$

Dans le code : `dt = np.random.exponential(1 / Lambda)` où `Lambda = np.sum(taux)`.

3. Fonctionnement de la Simulation ($N = 100$) - Part 2

2. Sélection de l'événement

Une fois le temps écoulé, on tire au sort l'événement qui se réalise. La probabilité que l'événement j soit choisi est :

$$\mathbb{P}(\text{événement } j) = \frac{r_j}{\sum r_k}$$

Dans le code : `np.random.choice(len(events), p=taux/Lambda)`.

Résultats n=1

Comparaison des probabilités de présence entre le modèle théorique et la simulation pour chaque station.

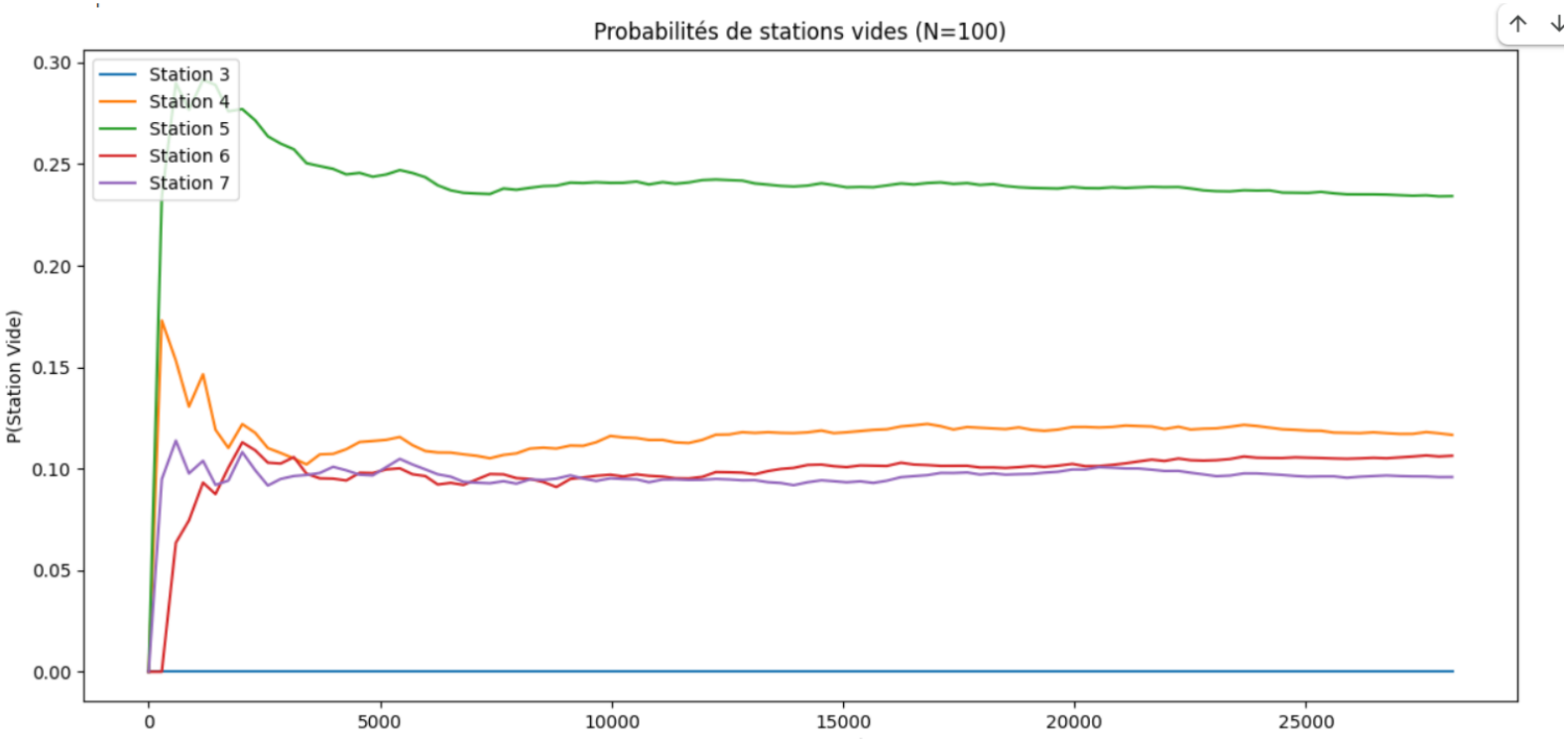
Station	**Théorique**	**Simulation**	**Écart**
Station 3	0.8216	0.8213	0.000349
Station 4	0.8419	0.8419	0.000035
Station 5	0.8632	0.8634	0.000125
Station 6	0.8417	0.8419	0.000112
Station 7	0.8381	0.8383	0.000201

Analyse

On observe une excellente corrélation entre les résultats théoriques et les données simulées (erreurs d'ordre 10^{-4} ou 10^{-5}).

Résultats n=100

Station	3	4	5	6	7
P(Station Vide)	0.0000	0.1174	0.2347	0.1066	0.0961



4. Les Limites de notre Modèle

Un éloignement de la réalité physique

Pour simplifier les calculs, plusieurs contraintes réelles ont été ignorées.

- **Capacité infinie :** Dans la vraie vie, une station Vélib pleine empêche les usagers de déposer leur vélo.
- **Absence de file d'attente :** Un usager réel peut accepter de patienter quelques minutes l'arrivée d'un vélo plutôt que d'abandonner immédiatement.
- **Intensités constantes :** λ et μ ne prennent pas en compte le cas jour/nuit (ex : flux massifs vers les bureaux le matin).

5. Difficultés Rencontrées

- **Problèmes Algorithmiques et Numériques :**
  - **Division par zéro :** Gérer les temps de trajets nuls (diagonale de la matrice T) en créant un masque binaire en Python (`mask = T_heures > 0`).
  - **Erreurs d'arrondi :** Lors du calcul des vecteurs propres pour la théorie, Python génère des erreurs de précision. Nous avons dû filtrer la valeur propre la plus proche de 1 (`np.isclose`).

6. Bilan : Ce que ce sujet nous inspire

Au-delà des mathématiques

Ce projet illustre parfaitement la difficulté de créer un système de transport en libre-service.

- On pourrait imaginer un modèle où l'application Vélib incite financièrement les utilisateurs (ex : bonus de temps) à ramener leurs vélos vers les stations ayant une forte probabilité qu'elle soit vide